Lösungen – Grundlagen der Programmierung

Python · DHBW Heilbronn · Wirtschaftsinformatik, 1. Semester · Dr. Markus Rosenfelder

Klausur laut Lernplan: Fr, 11.12.2026, 120 Minuten · Stoff: VL1–VL5 (Stand 21.10.2026) · inoffizielles Übungsmaterial, es gibt keine Musterklausur

Musterlösungen, erwartete Ausgaben, Trace-Tabellen und Punktvergabe (Erwartungshorizont) zu beiden Probeklausuren und zum Aufgabenpool.

Geprüft: Alle Programme, Ausgaben und Fehlermeldungen in dieser Datei wurden beim Erstellen tatsächlich mit **Python 3.14** (`python3.14 -I`) ausgeführt und automatisch eingefügt; Dateipfade in Tracebacks sind auf den Dateinamen gekürzt. pytest war in der Prüfumgebung nicht installiert: Die Testdateien wurden stattdessen durch direkten Aufruf aller `test_`-Funktionen geprüft. Die pytest-Ausgabe in Aufgabe B9 ist dem Format der VL04-Folie nachgebildet.

Inhalt

1. Probeklausur A
2. Probeklausur B
3. Aufgabenpool VL1: Einführung: Algorithmen, Datenstrukturen, Python-Werkzeuge (6 Aufgaben)
4. Aufgabenpool VL2: Variablen, Datentypen und Operatoren (7 Aufgaben)
5. Aufgabenpool VL3: Kontrollstrukturen und Listen (8 Aufgaben)
6. Aufgabenpool VL4: Funktionen, Rekursion und Tests (8 Aufgaben)
7. Aufgabenpool VL5: Objektorientierte Programmierung I (7 Aufgaben)

> Offen ist, ob die Klausur auf Papier oder am Rechner geschrieben wird. Die Probeklausuren sind als Papierklausur gestaltet; am Rechner lassen sich dieselben Aufgaben im Texteditor lösen und mit `uv run`, `ruff check` und `ty check` prüfen. Hilfsmittel (z. B. ein Spickzettel) sind nicht bekannt – angenommen ist „keine“.

Abgedeckte Themen

- VL1: Algorithmusbegriff und Eigenschaften, Datenstrukturen, Interpreter/Compiler/Zwischencode, Python-Syntax, uv/ruff/ty
- VL2: Datentypen, Variablen, `Final`, Operatoren (`/ // % **`), Strings, f-Strings, Typumwandlung, `Decimal`, Syntax vs. Semantik
- VL3: `if/elif/else`, `match`, bedingter Ausdruck, truthy/falsy, Kurzschluss, `while`, `for`/`range`, `break`/`continue`, Listen, `enumerate`, `zip`, Matrix, Tupel
- VL4: Funktionen, Typannotationen, Docstrings, `main()`, `return`/`None`, Gültigkeit (LEGB, `UnboundLocalError`), Parameterübergabe, Standardwerte, `*args`, Rekursion, `assert`, pytest
- VL5: Klassen, Objekte, `__init__`, `self`, Methoden, `__str__`/`__repr__`, Identität, Klassenattribute, Kapselung, `@property`, Hat-ein-Beziehung, `@dataclass`

Probeklausur A – Lösungen und Erwartungshorizont

Lösung A1: Multiple Choice — 12 P

Frage	richtig	Begründung
a	2. Option	CPython übersetzt in Bytecode, die Python-VM führt ihn aus (Variante Zwischencode, VL1).
b	`-4`	`//` rundet ab (Richtung $-\infty$), Java liefert -3. Geprüft: -4
c	`0.0`	Falsy sind `False, None, 0, 0.0, "", []`; `"0"`, `[0]`, `-1` sind wahr.
d	2, 5, 8	`stop` (11) gehört nicht dazu. Geprüft: `[2, 5, 8]`
e	2. Option	1. = terminierend, 3. = deterministisch, 4. = Effizienz.
f	`Konto.einzahlen(anna, 50)`	Das Objekt vor dem Punkt wird als erstes Argument (`self`) übergeben.

Erwartungshorizont

- je 2 P pro richtig angekreuzter Frage, 0 P bei falschem oder mehrfachem Kreuz

Lösung A2: Ausdrücke auswerten — 10 P

	Ausdruck	Wert	Typ
a)	`7 / 2`	`3.5`	`float`
b)	`2 + 3 * 4 ** 2`	`50`	`int`
c)	`int(9.99)`	`9`	`int`
d)	`"Py" * 2 + "!"`	`'PyPy!'`	`str`
e)	`"Heilbronn"[1:4]`	`'eil'`	`str`
f)	`len("DHBW Heilbronn")`	`14`	`int`
g)	`17 % 5 == 2 and not False`	`True`	`bool`
h)	`True + 1`	`2`	`int`
i)	`5 + 2.0`	`7.0`	`float`
j)	`"30" == 30`	`False`	`bool`

Hinweise: `/` liefert immer `float`; `**` vor `*` vor `+` ($2 + 3 \cdot 16 = 50$); `int()` schneidet ab; Slicing `[1:4]` nimmt Index 1 bis 3; `17 % 5` ist 2; `True` zählt als 1; `int + float` ergibt `float`; `str` und `int` sind nie gleich.

Erwartungshorizont

- je ½ P für richtigen Wert und ½ P für richtigen Typ
- Strings ohne Anführungszeichen: Wert trotzdem werten, wenn der Typ `str` richtig angegeben ist

Lösung A3: Was gibt dieser Code aus? 10 P

a)

Ausgabe (geprüft mit Python 3.14):

```
1 2 4 5 7 Ende
```

3, 6 werden mit `continue` übersprungen, bei 8 beendet `break` die Schleife, danach läuft `print("Ende")`.

b)

Ausgabe (geprüft mit Python 3.14):

```
5 [99, 2, 3, 4]
```

Python übergibt Namen, keine Kopien: `liste` zeigt zunächst auf dasselbe Listenobjekt wie `werte`. `liste[0] = 99` und `append(4)` verändern dieses Objekt (Nebeneffekt, außen sichtbar). `liste = [7, 7, 7]` bindet nur das lokale Namensschild neu; das `append(5)` betrifft die neue Liste. `zahl = zahl + 1` bindet ebenfalls nur lokal neu, und `int` ist unveränderlich – außen bleibt 5.

Erwartungshorizont

- a) 4 P: alle Zahlen in einer Zeile richtig (1 2 4 5 7) 3 P, „Ende“ in derselben Zeile 1 P; je falscher/fehlender Zahl −1 P
- b) Ausgabe 3 P: `5` 1 P, `[99, 2, 3, 4]` 2 P (nur `[99, 2, 3]`: 1 P)
- b) Begründung 3 P: gleiches Objekt / Verweis 1 P, Veränderung des Objekts vs. Neubindung 1 P, int unveränderlich bzw. nur lokale Neubindung 1 P

Lösung A4: Trace-Tabelle 10 P

Durchlauf	zahl % 10	summe	zahl	stellen	zahl > 0?
Start	–	0	4271	0	wahr
1	1	1	427	1	wahr
2	7	8	42	2	wahr
3	2	10	4	3	wahr
4	4	14	0	4	falsch → Ende

Selbst nachverfolgt (Zwischenausgabe je Durchlauf: zahl, summe, stellen):

```
Start 4271 0 0
   427 1 1
   42 8 2
   4 10 3
   0 14 4
4 Stellen, Quersumme 14
```

b)

Ausgabe (geprüft mit Python 3.14):

```
4 Stellen, Quersumme 14
```

c)

Ausgabe (geprüft mit Python 3.14):

```
0 Stellen, Quersumme 0
```

`while` prüft die Bedingung **vor** dem ersten Durchlauf. `0 > 0` ist falsch, der Rumpf läuft kein einziges Mal, alle Werte bleiben beim Startwert 0. (Fachlich stimmt das Ergebnis nicht ganz: Die Zahl 0 hat eine Stelle.)

Erwartungshorizont

- a) 6 P: je Durchlauf 1,5 P (alle vier Werte richtig), Bedingungsspalte mit „falsch“ am Ende nötig; Folgefehler werden nur einmal abgezogen
- b) 1 P: `4 Stellen, Quersumme 14`
- c) 3 P: Ausgabe 1 P, „Bedingung wird vorher geprüft, Schleife läuft nie“ 2 P

Lösung A5: Fehler finden und korrigieren — 14 P

Nr.	Zeile	Fehlerart	Korrektur
1	24	Syntaxfehler: fehlender Doppelpunkt	`if betrag > RABATT_GRENZE:`
2	22	Laufzeitfehler `TypeError`: `str + int`	`print(f"Rabatt: {{prozent}} %")` oder `str(prozent)`
3	16	Laufzeitfehler `UnboundLocalError`: Zuweisung (`+=`) macht den Namen lokal, er wird vor der Zuweisung gelesen	Zähler als Parameter übergeben und neuen Wert zurückgeben: `def zaehle_bestellung(anzahl: int) -> int: return anzahl + 1`; in `main`: `bestellungen_gesamt = zaehle_bestellung(bestellungen_gesamt)` mit lokaler Variable `bestellungen_gesamt = 0`
4	8–11	Logikfehler: `elif` prüft nacheinander, 120 trifft schon `>= 50` → 5 % statt 10 %	größte Grenze zuerst: `if betrag >= 100: return 10`, dann `elif betrag >= 50: return 5`

Reale Fehlermeldungen (Python 3.14, Pfade gekürzt)

Unverändertes Programm – der Syntaxfehler wird **vor dem Start** gemeldet, keine Zeile läuft:

```
  File "a5_bestellung.py", line 24
    if betrag > RABATT_GRENZE
                             ^
SyntaxError: expected ':'
```

Nach Korrektur von Fehler 1:

```
Traceback (most recent call last):
  File "a5_stufe2.py", line 29, in <module>
    main()
    ~~~~^^
  File "a5_stufe2.py", line 22, in main
    print("Rabatt: " + prozent + " %")
          ~~~~~~~~~~^~~~~~~~~
TypeError: can only concatenate str (not "int") to str
```

Nach Korrektur von Fehler 1 und 2 (die Ausgabe „Rabatt: 5 %“ zeigt den Logikfehler):

```
Rabatt: 5 %
Traceback (most recent call last):
  File "a5_stufe3.py", line 29, in <module>
    main()
    ~~~~^^
  File "a5_stufe3.py", line 23, in main
    zaehle_bestellung()
    ~~~~~~~~~~~~~~~~~^^
  File "a5_stufe3.py", line 16, in zaehle_bestellung
    bestellungen_gesamt += 1
    ^^^^^^^^^^^^^^^^^^^
UnboundLocalError: cannot access local variable 'bestellungen_gesamt' where it is not
associated with a value
```

Vollständig korrigierte Fassung:

```
from typing import Final

RABATT_GRENZE: Final = 100


def rabatt_prozent(betrag: float) -> int:
    if betrag >= 100:
        return 10
    elif betrag >= 50:
        return 5
    return 0


def zaehle_bestellung(anzahl: int) -> int:
    return anzahl + 1


def main() -> None:
    bestellungen_gesamt = 0
    betrag = 120.0
    prozent = rabatt_prozent(betrag)
    print(f"Rabatt: {prozent} %")
    bestellungen_gesamt = zaehle_bestellung(bestellungen_gesamt)
    if betrag > RABATT_GRENZE:
        print("Großbestellung")
    print(bestellungen_gesamt)


if __name__ == "__main__":
    main()
    assert rabatt_prozent(120.0) == 10
    assert rabatt_prozent(50) == 5
    assert rabatt_prozent(49.99) == 0
    assert rabatt_prozent(100) == 10
```

Ausgabe (geprüft mit Python 3.14):

```
Rabatt: 10 %
Großbestellung
1
```

b) Zuerst der **Syntaxfehler** (Zeile 24), weil Python die ganze Datei vor dem Ausführen übersetzt. Der **Logikfehler** (Rabattreihenfolge) erzeugt nie eine Fehlermeldung – nur ein falsches Ergebnis.

Erwartungshorizont

- a) je Fehler 3 P: Zeile 1 P, Fehlerart (bei Laufzeitfehlern mit Exception-Namen) 1 P, korrekte Korrektur 1 P
- Fehler 3: auch andere korrekte Lösungen (Zähler in `main` führen) zählen; nur „Zeile löschen“ gibt 0 P für die Korrektur
- b) 2 P: Syntaxfehler zuerst 1 P, Logikfehler nie gemeldet 1 P

Lösung A6: Lückencode mit `match` — 10 P

Lücke	Lösung
(1)	`float | None`
(2)	`match`
(3)	`case "EU" | "CH"`
(4)	`case _`
(5)	`zip(zonen, gewichte, strict=True)`
(6)	`preis is None`
(7)	`preis:.2f`

```
def versandkosten(zone: str, gewicht_kg: float) -> float | None:
    match zone:
        case "DE":
            kosten = 4.90
        case "EU" | "CH":
            kosten = 9.90
        case _:
            return None
    if gewicht_kg > 5:
        kosten += 1.50
    return kosten


def main() -> None:
    zonen = ["DE", "CH", "US"]
    gewichte = [2.0, 7.5, 1.0]
    for zone, gewicht in zip(zonen, gewichte, strict=True):
        preis = versandkosten(zone, gewicht)
        if preis is None:
            print(f"{zone}: nicht lieferbar")
        else:
            print(f"{zone}: {preis:.2f} Euro")


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
DE: 4.90 Euro
CH: 11.40 Euro
US: nicht lieferbar
```

CH: 9.90 + 1.50 = 11.40, weil 7,5 kg > 5 kg.

Erwartungshorizont

- je Lücke 1 P (7 P); bei (5) ohne `strict=True` volle Punktzahl, (6) `preis == None` ½ P
- Ausgabe 3 P: je Zeile 1 P; „11.4“ statt „11.40“ ½ P Abzug

Lösung A7: Funktion schreiben — 12 P

```
def bestellwert(preise: list[float], mengen: list[int], rabatt: float = 0.0) -> float:
    """Berechnet den Wert einer Bestellung nach Rabatt.

    Args:
        preise: Stückpreise der Positionen in Euro.
        mengen: Bestellmengen, passend zu preise.
        rabatt: Rabatt als Anteil zwischen 0 und 0.5, Standard 0.0.

    Returns:
        Den Bestellwert in Euro, auf zwei Nachkommastellen gerundet.
    """
    if rabatt < 0 or rabatt > 0.5:
        rabatt = 0.0
    summe = 0.0
    for preis, menge in zip(preise, mengen, strict=True):
        summe += preis * menge
    return round(summe * (1 - rabatt), 2)


def main() -> None:
    print(bestellwert([2.5, 10.0], [4, 3]))
    print(bestellwert([2.5, 10.0], [4, 3], 0.1))
    print(bestellwert([2.5, 10.0], [4, 3], 0.8))
    print(bestellwert(mengen=[4, 3], preise=[2.5, 10.0], rabatt=0.1))
    print(bestellwert([], []))
    print(bestellwert([19.99], [3], 0.5))


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
40.0
36.0
40.0
36.0
0.0
29.98
```

b) `bestellwert(mengen=[4, 3], preise=[2.5, 10.0], rabatt=0.1)` → `36.0` (vierte Zeile der Ausgabe oben).

Alternative ohne `zip`: `for i in range(len(preise)): summe += preise[i] * mengen[i]`. Mit `strict=True` meldet Python ungleich lange Listen: `ValueError: zip() argument 2 is longer than argument 1`

Erwartungshorizont

- Signatur mit allen Typannotationen und Standardwert `rabatt: float = 0.0`: 2 P
- Docstring im Google-Stil mit `Args:` und `Returns:`: 2 P
- Summe Preis × Menge über alle Positionen (Schleife korrekt, Startwert 0): 3 P
- Prüfung des ungültigen Rabatts (< 0 oder > 0.5) mit Rückfall auf 0: 2 P
- `return round(..., 2)` mit Rabattabzug: 1 P
- b) 2 P: Schlüsselwortargumente in geforderter Reihenfolge 1 P, Ergebnis 36.0 1 P

Lösung A8: Rekursion **12 P**

a) Abstieg `r(5) → r(3) → r(1)`; `r(1) = 1` (Abbruch, da 1 ≤ 1). Aufstieg: `r(3) = 3 + 2·1 = 5`, `r(5) = 5 + 2·5 = 15`.
Abstieg `r(6) → r(4) → r(2) → r(0)`; `r(0) = 1`. Aufstieg: `r(2) = 2 + 2·1 = 4`, `r(4) = 4 + 2·4 = 12`, `r(6) = 6 + 2·12 = 30`.

Ausgabe (geprüft mit Python 3.14):

```
15
30
```

b)

```
def quersumme(n: int) -> int:
    """Berechnet die Quersumme einer nicht negativen ganzen Zahl rekursiv.

    Args:
        n: Nicht negative ganze Zahl.

    Returns:
        Die Summe der Ziffern von n.
    """
    if n < 10:
        return n
    return n % 10 + quersumme(n // 10)


def main() -> None:
    print(quersumme(4271), quersumme(7), quersumme(0), quersumme(1000))


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
14 7 0 1
```

c) Die Abbruchbedingung `n == 0` wird nie erreicht (−1, −2, −3, …). Der Aufrufstapel wächst, bis Python abbricht (letzte Zeilen der echten Meldung):

```
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Robuster: `if n <= 0: return 1` (oder negative Werte vorher abweisen).

Erwartungshorizont

- a) 4 P: r(5) = 15 mit Aufrufkette 2 P, r(6) = 30 mit Aufrufkette 2 P (nur Ergebnis: je 1 P)
- b) 6 P: Signatur mit Typen 1 P, korrekte Abbruchbedingung (`n < 10` oder `n == 0` → `0`) 2 P, rekursiver Aufruf mit `n // 10` und `n % 10` 2 P, Kommentare Abbruch/Rekursion 1 P
- Schleife oder `str()` verwendet: höchstens 2 P
- c) 2 P: RecursionError bzw. Abbruch nie erreicht 1 P, Korrektur `n <= 0` 1 P

Lösung A9: Tests mit pytest — 8 P

Dateiname: `test_bestellung.py` (pytest findet nur Dateien `test_*.py` und Funktionen `test_…`).

```
from bestellung import bestellwert


def test_ohne_rabatt() -> None:
    assert bestellwert([2.5, 10.0], [4, 3]) == 40.0


def test_mit_rabatt() -> None:
    assert bestellwert([2.5, 10.0], [4, 3], 0.1) == 36.0


def test_ungueltiger_rabatt() -> None:
    assert bestellwert([2.5, 10.0], [4, 3], 0.8) == 40.0
    assert bestellwert([2.5, 10.0], [4, 3], -0.1) == 40.0


def test_leere_bestellung() -> None:
    assert bestellwert([], []) == 0.0


def test_grenze_halber_rabatt() -> None:
    assert bestellwert([10.0], [2], 0.5) == 10.0
```

Geprüft ohne pytest (in dieser Umgebung nicht installiert) durch Aufruf aller `test_`-Funktionen:
`test_bestellung: 5 Tests bestanden`

b) `uv run pytest` (einmalig vorher `uv add --dev pytest`). `.F.`: drei Tests in der Datei, der erste und dritte bestanden (`.`), der zweite fehlgeschlagen (`F`).

Erwartungshorizont

- a) 6 P: Import `from bestellung import bestellwert` 1 P, drei korrekt benannte Testfunktionen mit `assert` je 1 P, sinnvoller Randfall 1 P, korrekte Erwartungswerte und Dateiname 1 P
- Vergleich gerundeter Werte mit `==` ist hier zulässig, weil die Funktion selbst rundet
- b) 2 P: Befehl 1 P, Bedeutung von `./F` 1 P

Lösung A10: Klassen und Objekte **14 P**

a)

```
from decimal import Decimal


class Artikel:
    anzahl = 0

    def __init__(self, name: str, preis: Decimal, bestand: int = 0) -> None:
        Artikel.anzahl += 1
        self.nummer = f"A{Artikel.anzahl:03}"
        self.name = name
        self._preis = Decimal(0)
        self.preis = preis
        self.bestand = bestand

    @property
    def preis(self) -> Decimal:
        return self._preis

    @preis.setter
    def preis(self, neuer_preis: Decimal) -> None:
        if neuer_preis < 0:
            print(f"Ungültiger Preis: {neuer_preis}")
            return
        self._preis = neuer_preis

    def verkaufen(self, menge: int) -> bool:
        if menge <= 0 or menge > self.bestand:
            print(f"Verkauf von {menge} x {self.name} nicht möglich.")
            return False
        self.bestand -= menge
        return True

    def __str__(self) -> str:
        return f"{self.nummer} {self.name}: {self.preis} Euro, Bestand {self.bestand}"


def main() -> None:
    kaffee = Artikel("Kaffee 1 kg", Decimal("12.99"), 40)
    tee = Artikel("Grüner Tee", Decimal("3.49"))
    print(kaffee)
    print(tee)
    print(kaffee.verkaufen(5), kaffee.verkaufen(100), tee.verkaufen(1))
    kaffee.preis = Decimal("-5")
    print(kaffee)
    print(Artikel.anzahl)


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
A001 Kaffee 1 kg: 12.99 Euro, Bestand 40
A002 Grüner Tee: 3.49 Euro, Bestand 0
Verkauf von 100 x Kaffee 1 kg nicht möglich.
Verkauf von 1 x Grüner Tee nicht möglich.
True False False
Ungültiger Preis: -5
A001 Kaffee 1 kg: 12.99 Euro, Bestand 35
2
```

b)

Ausgabe (geprüft mit Python 3.14):

```
a c c
2 2 12
True False
```

Erklärung: `c = b` erzeugt kein neues Objekt, also ändert `c.name` auch `b`. `b.stand += 10` liest das Klassenattribut (2) und legt ein **Instanzattribut** `stand = 12` nur für `b` an; `Zaehler.stand` und `a.stand` bleiben 2. `b is c` ist `True`; `==` vergleicht bei normalen Klassen die Identität → `False` (das neue Objekt erhöht den Zähler danach auf 3).

Erwartungshorizont

- a) 10 P: Klassenattribut und Zählen über `Artikel.anzahl` 1 P, Nummer mit `f"A{{Artikel.anzahl:03}}"` 1 P, `__init__` mit Typen, Standardwert und `self.`-Attributen 2 P, Property mit `@property` 1 P, Setter `@preis.setter` mit Prüfung und Rückkehr ohne Änderung 2 P, Setter-Nutzung in `__init__` 1 P, `verkaufen` mit beiden Prüfungen und Rückgabewerten 1 P, `__str__` 1 P
- Klassenattribut über `self.anzahl += 1` erhöht: −1 P (legt Instanzattribut an)
- b) 4 P: Zeile 1 1 P, Zeile 2 2 P (je falscher Zahl −1), Zeile 3 1 P

Lösung A11: Konzeptfragen — 8 P

a. **Parameter** ist der Platzhalter in der Definition (`def begruessung(name: str)` → `name`), **Argument** der konkrete Wert beim Aufruf (`begruessung("Bob")` → `"Bob"`).

b. Vorteil: elegant und kurz bei selbstähnlichen Problemen, mathematische Definition direkt umsetzbar, wenig Zustand. Nachteil: speicherintensiv (ein Rahmen pro Aufruf, `RecursionError` ab ca. 1000 Ebenen), langsamer, schwerer zu debuggen.

c. Kapselung: Ein Objekt schützt seinen inneren Zustand und bestimmt selbst, wie er geändert wird; nach außen nur das Nötige. Mittel in Python: Unterstrich-Konvention (`_kontostand`) und Properties (`@property`, ggf. mit Setter, der prüft).

d. `__name__` ist nur beim direkten Start `"__main__"`. Wird die Datei importiert (z. B. von einer Testdatei), läuft `main()` nicht los – die Funktionen lassen sich wiederverwenden und testen.

Erwartungshorizont

- je Teilaufgabe 2 P: Kernaussage 1 P, Beispiel/zweiter Aspekt 1 P

Probeklausur B – Lösungen und Erwartungshorizont

Lösung B1: Multiple Choice — 12 P

Frage	richtig	Begründung
a	Stapel (Stack, LIFO)	Die zuletzt besuchte Seite kommt zuerst zurück.
b	dynamisch, aber stark typisiert	Werte haben feste Typen, Variablen nicht; Python rät nicht (`"1" + 1` → `TypeError`).
c	`False`	`float` ist binär gespeichert und ungenau. Geprüft: `False`
d	`a or (b and (not c))`	Rangfolge: Vergleiche → `not` → `and` → `or`.
e	`None`	Geprüft: `print(f())` → `None`
f	`__init__`, `__repr__`, `__eq__`	Laut Folie VL05; `==` vergleicht dann die Werte.

Erwartungshorizont

- je 2 P pro richtig angekreuzter Frage

Lösung B2: Ausdrücke auswerten — 10 P

	Ausdruck	Wert	Typ
a)	`17 // 5`	`3`	`int`
b)	`17 % 5`	`2`	`int`
c)	`6 / 3`	`2.0`	`float`
d)	`int("42") + 1`	`43`	`int`
e)	`float(5)`	`5.0`	`float`
f)	`"Hallo"[-1]`	`'o'`	`str`
g)	`"ll" in "Hallo"`	`True`	`bool`
h)	`18 <= 17 <= 65`	`False`	`bool`
i)	`[0] * 3`	`[0, 0, 0]`	`list`
j)	`round(2.675, 2)`	`2.67`	`float`

`round(2.675, 2)` ergibt 2.67, weil 2.675 als `float` minimal kleiner gespeichert ist (Folie VL02). `18 <= 17 <= 65` ist ein verketteter Vergleich: schon `18 <= 17` ist falsch.

Erwartungshorizont

- je ½ P für Wert und ½ P für Typ

Lösung B3: Was gibt dieser Code aus? — 10 P

a)

Ausgabe (geprüft mit Python 3.14):

```
2
4 6
Summe: 12
```

Nur gerade Werte werden ausgegeben (mit `end=" "` in einer Zeile); das `print()` nach jeder Zeile der Matrix erzeugt den Zeilenumbruch. Summe 2 + 4 + 6 = 12.

b)

Ausgabe (geprüft mit Python 3.14):

```
prüfe 4
prüfe 8
2
prüfe 4
prüfe 8
-1
```

`return i` beendet die ganze Funktion sofort, also wird 15 nicht mehr „geprüft“. Im zweiten Aufruf wird nichts gefunden: Nach der Schleife kommt `return -1`.

Erwartungshorizont

- a) 5 P: Zeile „2“ 1 P, Zeile „4 6“ 2 P, „Summe: 12“ 1 P, richtige Zeilenumbrüche 1 P
- b) 5 P: erster Aufruf (prüfe 4, prüfe 8, 2) 3 P, zweiter Aufruf (prüfe 4, prüfe 8, −1) 2 P; „prüfe 15“ ausgegeben: −1 P

Lösung B4: Trace-Tabelle — 10 P

Durchlauf	a % b	a	b	schritte	b != 0?
Start	–	252	105	0	wahr
1	42	105	42	1	wahr
2	21	42	21	2	wahr
3	0	21	0	3	falsch → Ende

Selbst nachverfolgt mit Zwischenausgaben (a, b, schritte):

```
start 252 105 0
   a % b = 42
   105 42 1
   a % b = 21
   42 21 2
   a % b = 0
   21 0 3
```

b)

Ausgabe (geprüft mit Python 3.14):

```
ggT = 21 nach 3 Schritten
```

c)

Ausgabe (geprüft mit Python 3.14):

```
ggT = 105 nach 1 Schritten
```

Bei der Tupel-Zuweisung wird die rechte Seite **vollständig** mit den alten Werten berechnet, bevor zugewiesen wird. In der Variante ist `a` beim Rechnen von `a % b` schon gleich `b`, also gilt `b`

`% b = 0`: Die Schleife endet nach einem Durchlauf mit dem falschen Ergebnis 105.

Erwartungshorizont

- a) 6 P: je Durchlauf 2 P (Folgefehler nur einmal abziehen)
- b) 1 P
- c) 3 P: Ausgabe 1 P, Erklärung „rechte Seite zuerst / b % b = 0“ 2 P

Lösung B5: Fehler finden und korrigieren — 14 P

Nr.	Zeile	Fehlerart	Korrektur
1	1–3	fehlendes `return` im Zweig der leeren Liste → implizit `None`; in Zeile 29 dann `TypeError` (`None + 1`)	nach dem `if`: `return 0.0`
2	6	Logikfehler: veränderbarer Standardwert `[]` wird nur einmal beim Definieren erzeugt und zwischen Aufrufen geteilt (ruff B006)	`verlauf: list[float] | None = None`, im Rumpf `if verlauf is None: verlauf = []`
3	12–16	Laufzeitfehler `UnboundLocalError`: bei 0 ≤ t ≤ 30 wird `hinweis` nie zugewiesen	`else: hinweis = "normal"` ergänzen
4	20	Laufzeitfehler `TypeError`: `/` liefert immer `float`, `range` braucht `int`	`range(int(wert // 10))` oder `range(int(wert) // 10)`

Reale Ausgaben und Fehlermeldungen (Python 3.14, Pfade gekürzt)

Unverändertes Programm:

```
[21.5]
[21.5, 19.0]
Traceback (most recent call last):
  File "b5_messwerte.py", line 34, in <module>
    main()
    ~~~~^^
  File "b5_messwerte.py", line 28, in main
    print(bewerte(20.0))
          ~~~~~~~^^^^^^
  File "b5_messwerte.py", line 16, in bewerte
    return hinweis
           ^^^^^^^
UnboundLocalError: cannot access local variable 'hinweis' where it is not associated with a
value
```

Nach Korrektur von Fehler 3 (Zeilennummern verschieben sich um die eingefügten Zeilen):

```
[21.5]
[21.5, 19.0]
normal
Traceback (most recent call last):
  File "b5_stufe2.py", line 36, in <module>
    main()
    ~~~~^^
  File "b5_stufe2.py", line 31, in main
    print(mittelwert([]) + 1)
          ~~~~~~~~~~~~~~^~~
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

Zusätzlich Fehler 1 korrigiert:

```
[21.5]
[21.5, 19.0]
normal
1.0
Traceback (most recent call last):
  File "b5_stufe3.py", line 37, in <module>
    main()
    ~~~~^^
  File "b5_stufe3.py", line 33, in main
    drucke_balken(35.0)
    ~~~~~~~~~~~~~^^^^^^
  File "b5_stufe3.py", line 23, in drucke_balken
    for _ in range(wert / 10):
             ~~~~~^^^^^^^^^^^
TypeError: 'float' object cannot be interpreted as an integer
```

Vollständig korrigiert:

```
def mittelwert(werte: list[float]) -> float:
    if len(werte) > 0:
        return sum(werte) / len(werte)
    return 0.0


def merke(wert: float, verlauf: list[float] | None = None) -> list[float]:
    if verlauf is None:
        verlauf = []
    verlauf.append(wert)
    return verlauf


def bewerte(temperatur: float) -> str:
    if temperatur > 30:
        hinweis = "heiß"
    elif temperatur < 0:
        hinweis = "Frost"
    else:
        hinweis = "normal"
    return hinweis


def drucke_balken(wert: float) -> None:
    for _ in range(int(wert // 10)):
        print("#", end="")
    print()


def main() -> None:
    print(merke(21.5))
    print(merke(19.0))  # erwartet: [19.0]
    print(bewerte(20.0))
    print(mittelwert([]) + 1)
    drucke_balken(35.0)


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
[21.5]
[19.0]
normal
1.0
###
```

b) Zuerst erscheinen `[21.5]` und `[21.5, 19.0]`, dann der `UnboundLocalError` aus `bewerte`. Der Fehler in `merke` ist ein **Logikfehler**: Der Code ist gültig und läuft, er liefert nur ein falsches Ergebnis (`[21.5, 19.0]` statt `[19.0]`).

Erwartungshorizont

- a) je Fehler 3 P: Stelle 1 P, Fehlerart/Ursache 1 P, Korrektur 1 P
- Fehler 1 auch als Rückgabetyp `float | None` plus Prüfung beim Aufrufer akzeptieren
- b) 2 P: erste Ausgabe(n) und UnboundLocalError 1 P, Begründung Logikfehler 1 P

Lösung B6: Lückencode: Klasse mit Property — 10 P

Lücke	Lösung
(1)	`anzahl`
(2)	`self`
(3)	`Konto.anzahl`
(4)	`self.kontostand` (nicht `self._kontostand`, sonst wird der Setter umgangen)
(5)	`@property`
(6)	`@kontostand.setter`
(7)	`f"{{self.nummer}} {{self.inhaber}}: {{self.kontostand}} Euro"`

Ausgabe (geprüft mit Python 3.14):

```
Ungültig: negativer Kontostand.
K001 Anna: 100.00 Euro
K002 Ben: 0 Euro
Ungültig: negativer Kontostand.
100.00 20
```

Ben wird mit −5 angelegt: Der Setter weist den Wert ab, es bleibt `Decimal(0)`, das als `0` ausgegeben wird. `anna.kontostand -= Decimal("150")` ruft erst den Getter, dann den Setter mit −50 auf → abgewiesen, Annas Stand bleibt 100.00.

Erwartungshorizont

- je Lücke 1 P (7 P)
- Ausgabe 3 P: beide Meldungen „Ungültig …“ an der richtigen Stelle 1 P, die zwei `print`-Zeilen der Objekte 1 P, letzte Zeile 1 P

Lösung B7: Funktion schreiben: Nachbestellung 12 P

```
def nachbestellmengen(
    bestaende: list[int], meldebestaende: list[int], hoechstbestaende: list[int]
) -> list[int]:
    """Berechnet für jeden Artikel die Nachbestellmenge.

    Liegt der Bestand unter dem Meldebestand, wird bis zum Höchstbestand
    aufgefüllt, sonst wird nichts bestellt (Menge 0).

    Args:
        bestaende: Aktuelle Bestände je Artikel.
        meldebestaende: Meldebestände je Artikel.
        hoechstbestaende: Höchstbestände je Artikel.

    Returns:
        Eine Liste mit der Bestellmenge je Artikel.
    """
    mengen: list[int] = []
    for bestand, melde, hoechst in zip(
        bestaende, meldebestaende, hoechstbestaende, strict=True
    ):
        if bestand < melde:
            mengen.append(hoechst - bestand)
        else:
            mengen.append(0)
    return mengen


def main() -> None:
    artikel = ["Schrauben M8", "Dichtringe", "Muttern M8"]
    mengen = nachbestellmengen([12, 45, 8], [20, 30, 25], [100, 150, 120])
    for name, menge in zip(artikel, mengen, strict=True):
        if menge > 0:
            print(f"Bestellung: {name} - {menge} Stück")


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
Bestellung: Schrauben M8 - 88 Stück
Bestellung: Muttern M8 - 112 Stück
```

Erwartungshorizont

- a) 8 P: Signatur mit Typannotationen 2 P, Docstring mit Args/Returns 2 P, Schleife über alle drei Listen (zip oder Index) 2 P, korrekte Bedingung „<“ und Menge 1 P, Rückgabe der Liste 1 P
- „<=“ statt „<“: −1 P (Folie: „kleiner als der Meldebestand“)
- b) 4 P: Funktionsaufruf und Schleife mit Namen 2 P, nur Mengen > 0 ausgeben 1 P, `if __name__ == "__main__": main()` 1 P

Lösung B8: Rekursion **12 P**

a) und c)

```
def umdrehen(text: str) -> str:
    """Dreht einen Text rekursiv um.

    Args:
        text: Beliebiger Text, auch leer.

    Returns:
        Den Text in umgekehrter Reihenfolge.
    """
    if text == "":
        return ""
    return umdrehen(text[1:]) + text[0]


def summe_liste(zahlen: list[int]) -> int:
    if len(zahlen) == 0:
        return 0
    return zahlen[0] + summe_liste(zahlen[1:])


def main() -> None:
    print(umdrehen("abc"), umdrehen("Heilbronn"), repr(umdrehen("")))
    print(summe_liste([4, 1, 5]), summe_liste([]))


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
cba nnorblieH ''
10 0
```

b)

Ebene	Aufruf	Rückgabewert
1	`umdrehen("abc")`	`umdrehen("bc") + "a"` = `"cba"`
2	`umdrehen("bc")`	`umdrehen("c") + "b"` = `"cb"`
3	`umdrehen("c")`	`umdrehen("") + "c"` = `"c"`
4	`umdrehen("")`	`""` (Abbruchbedingung)

c) Originalfunktion

Ausgabe (geprüft mit Python 3.14):

```
10
Traceback (most recent call last):
  File "b8_summe.py", line 8, in <module>
    print(summe_liste([]))
          ~~~~~~~~~~~~^^^^
  File "b8_summe.py", line 4, in summe_liste
    return zahlen[0] + summe_liste(zahlen[1:])
           ~~~~~~~^^^
IndexError: list index out of range
```

`summe_liste([4, 1, 5])` = 10. Bei `[]` ist `len(zahlen) == 1` falsch, also wird `zahlen[0]` gelesen → `IndexError`. Korrektur: `if len(zahlen) == 0: return 0` (dann deckt die Rekursion auch einelementige Listen ab).

Erwartungshorizont

- a) 5 P: Signatur 1 P, Abbruchbedingung leerer Text 2 P, rekursiver Aufruf mit `text[1:]` und Anhängen von `text[0]` 2 P
- b) 3 P: vier Ebenen 1 P, Rückgabewerte im Aufstieg 2 P
- c) 4 P: 10 1 P, IndexError 1 P, Begründung 1 P, Korrektur 1 P

Lösung B9: Tests mit pytest — 8 P

a)

```
from lager import nachbestellmengen


def test_beispiel_aus_der_vorlesung() -> None:
    assert nachbestellmengen([12, 45, 8], [20, 30, 25], [100, 150, 120]) == [88, 0, 112]


def test_grenze_bestand_gleich_meldebestand() -> None:
    assert nachbestellmengen([20], [20], [100]) == [0]


def test_leere_listen() -> None:
    assert nachbestellmengen([], [], []) == []


def test_bestand_null() -> None:
    assert nachbestellmengen([0], [5], [50]) == [50]
```

Geprüft ohne pytest (nicht installiert) durch Aufruf aller `test_`-Funktionen: `test_lager: 4 Tests bestanden`

b)

Echte Rückgabewerte der fehlerhaften Funktion (selbst ausgeführt):

```
test_potenz_null .
test_potenz_vier F
test_potenz_eins F
9.0 8.0 1.0
```

`test_potenz_null` besteht als einziger, weil bei Exponent 0 sofort die (richtige) Abbruchbedingung `return 1.0` greift – der fehlerhafte rekursive Zweig läuft gar nicht. Tatsächlich kam `9.0` heraus, erwartet war `16.0` (2 + 2 + 2 + 2 + 1). Fehler: `+` statt `*`. Korrektur: `return basis * potenz(basis, exponent - 1)`.

Ausgabe (geprüft mit Python 3.14):

```
16.0 7.0 1.0
```

Erwartungshorizont

- a) 4 P: Import 1 P, drei Testfunktionen mit korrekten Erwartungen 2 P (Grenze und leere Liste enthalten), Namen mit `test_` 1 P
- b) 4 P: bestandener Test und Grund 1 P, tatsächlich 9.0 / erwartet 16.0 1 P, Fehler erkannt 1 P, Korrektur 1 P

Lösung B10: Klassen, Hat-ein-Beziehung und `@dataclass` 14 P

```
from dataclasses import dataclass


class Mitarbeiter:
    anzahl = 0

    def __init__(self, name: str, monatsgehalt: int) -> None:
        Mitarbeiter.anzahl += 1
        self.personalnummer = f"M{Mitarbeiter.anzahl:03}"
        self.name = name
        self.monatsgehalt = monatsgehalt

    @property
    def jahresgehalt(self) -> int:
        return self.monatsgehalt * 12

    def gehalt_erhoehen(self, prozent: int) -> bool:
        if prozent < 1 or prozent > 20:
            print(f"Erhöhung um {prozent} % nicht erlaubt.")
            return False
        self.monatsgehalt = self.monatsgehalt * (100 + prozent) // 100
        return True

    def __str__(self) -> str:
        return f"{self.personalnummer} {self.name}: {self.monatsgehalt} Euro/Monat"


class Abteilung:
    def __init__(self, name: str) -> None:
        self.name = name
        self.mitarbeitende: list[Mitarbeiter] = []

    def aufnehmen(self, mitarbeiter: Mitarbeiter) -> None:
        self.mitarbeitende.append(mitarbeiter)

    def personalkosten(self) -> int:
        summe = 0
        for m in self.mitarbeitende:
            summe += m.jahresgehalt
        return summe


@dataclass
class Adresse:
    strasse: str
    plz: str
    ort: str


def main() -> None:
    lena = Mitarbeiter("Lena", 4200)
    tim = Mitarbeiter("Tim", 3800)
    print(lena)
    print(lena.jahresgehalt)
    print(lena.gehalt_erhoehen(5), lena.gehalt_erhoehen(50))
    print(lena, lena.jahresgehalt)
    it = Abteilung("IT")
    it.aufnehmen(lena)
    it.aufnehmen(tim)
    print(it.personalkosten())
    a1 = Adresse("Bildungscampus 4", "74076", "Heilbronn")
```

```
    a2 = Adresse("Bildungscampus 4", "74076", "Heilbronn")
    print(a1 == a2, a1 is a2)
    print(a1)
    try:
        lena.jahresgehalt = 1  # type: ignore
    except AttributeError as e:
        print("AttributeError:", e)


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
M001 Lena: 4200 Euro/Monat
50400
Erhöhung um 50 % nicht erlaubt.
True False
M001 Lena: 4410 Euro/Monat 52920
98520
True False
Adresse(strasse='Bildungscampus 4', plz='74076', ort='Heilbronn')
AttributeError: property 'jahresgehalt' of 'Mitarbeiter' object has no setter
```

Die letzte Zeile zeigt zusätzlich, dass die Property ohne Setter nur lesbar ist (gehört nicht zur Aufgabe). Hinweis: Die Meldung „Erhöhung um 50 % …“ steht vor `True False`, weil Python erst beide Argumente von `print` auswertet.

c) `a1 == a2` → `True` (dataclass vergleicht die Werte), `a1 is a2` → `False` (zwei Objekte). Bei einer normalen Klasse ohne `__eq__` vergleicht `==` die Identität → `False`.

Erwartungshorizont

- a) 7 P: Klassenattribut/Nummer 2 P, `__init__` 1 P, Property `jahresgehalt` ohne Setter 1 P, `gehalt_erhoehen` mit Grenzen, Rechnung und Rückgaben 2 P, `__str__` 1 P
- b) 4 P: leere Liste mit Typannotation `list[Mitarbeiter]` in `__init__` 1 P, `aufnehmen` 1 P, `personalkosten` mit Schleife über `jahresgehalt` 2 P
- Liste als Klassenattribut statt in `__init__`: −1 P (würde von allen Abteilungen geteilt)
- c) 3 P: dataclass korrekt 1 P, `True`/`False` 1 P, normale Klasse → Identität 1 P

Lösung B11: Konzeptfragen — 8 P

a. **Syntax** = Form/Grammatik (Fehler: `print("Hallo"` → `SyntaxError`, vor dem Start), **Semantik** = Bedeutung/Wirkung (Fehler: `5 + 2 / 2` statt `(5 + 2) / 2` – läuft, Ergebnis falsch).

b. Compiler: hohe Ausführungsgeschwindigkeit (alles vorab in Maschinencode). Interpreter: schnelles Ausprobieren, Änderungen sofort sichtbar, kein Kompilierschritt.

c. Imperativ: das „Wie“ – veränderbarer Zustand, Anweisungen in fester Reihenfolge, Schleifen. Applikativ: das „Was“ – Programme entstehen durch Anwendung von Funktionen ohne Nebenwirkungen, Rekursion statt Schleifen.

d. Das zweite `def` überschreibt das erste (ruff F811); ein Aufruf mit zwei Argumenten scheitert dann mit `TypeError: addiere() missing 1 required positional argument: 'c'`. Stattdessen: Standardwerte (`c: int = 0`), `*args` oder unterschiedliche Funktionsnamen.

Erwartungshorizont

- je Teilaufgabe 2 P: Kernaussage 1 P, Beispiel/Begründung 1 P

Aufgabenpool VL1 – Lösungen

Lösung 1.1: Eigenschaften von Algorithmen

a) deterministisch · b) terminierend · c) determiniert · d) Endlichkeit, Ausführbarkeit, Korrektheit, Effizienz.

Lösung 1.2: Datenstrukturen wählen

a) Warteschlange (FIFO) · b) Stapel (LIFO, die letzte Aktion wird zuerst zurückgenommen) · c) Baum (Hierarchie) · d) Graph (Knoten = Lager, Kanten = Routen) · e) Baum (Jahr → Lieferant → Monat), wie in der Musterlösung VL01.

Lösung 1.3: Nachbestell-Algorithmus durchspielen

Geprüft mit einem kleinen Programm (gleiche Regel):

```
artikel = [
    ("Kabelbinder", 5, 10, 50),
    ("Dübel", 30, 30, 80),
    ("Schrauben M8", 12, 20, 100),
    ("Unterlegscheiben", 0, 40, 200),
    ("Muttern M8", 41, 25, 120),
]
for name, bestand, melde, hoechst in artikel:
    if bestand < melde:
        print(f"{name}: {hoechst - bestand}")
```

Ausgabe (geprüft mit Python 3.14):

```
Kabelbinder: 45
Schrauben M8: 88
Unterlegscheiben: 200
```

Dübel: 30 ist **nicht kleiner** als 30 → keine Bestellung (Grenzfall). Muttern: 41 ≥ 25 → keine Bestellung.

Lösung 1.4: Einen Algorithmus aufschreiben

1. Antrag entgegennehmen: Zeitraum und Anzahl Urlaubstage ablesen.
2. Resturlaub nachsehen. Reicht er nicht: ablehnen, weiter mit 6.
3. Prüfen, ob der Zeitraum in einer Sperrzeit liegt. Ja: ablehnen, weiter mit 6.
4. Prüfen, ob eine Vertretung eingetragen ist. Nein: zurück an Antragsteller, weiter mit 6.
5. Antrag genehmigen, Resturlaub um die Tage verringern.
6. Entscheidung mitteilen. Fertig.

Jeder Zweig endet (Endlichkeit), jeder Schritt ist konkret ausführbar. „Antrag kurz ansehen“ ist nicht ausführbar/eindeutig: Was genau wird geprüft, und was passiert bei einer Abweichung? (vgl. Musterlösung VL01, Aufgabe 2)

Lösung 1.5: Vom Quellcode zum Programm

Strategie	Ablauf	Vorteil	Nachteil	Beispiel
Interpreter	Anweisung für Anweisung lesen und sofort ausführen	schnelles Ausprobieren	langsamere Ausführung	Shell-Skripte
Compiler	gesamter Code vorab in Maschinencode	hohe Performance	Kompilierschritt vor jedem Test, nur Zielplattform	C, C++, Rust, Go
Zwischencode	Bytecode, den eine virtuelle Maschine ausführt	portabel	Laufzeitumgebung nötig, längerer Start	Java, C#, Python

b) Zwischencode: CPython übersetzt in Bytecode, die Python-VM führt ihn aus. JIT (Just-in-Time): häufig genutzter Code wird zur Laufzeit in Maschinencode übersetzt (in Python seit 3.13 experimentell).

Lösung 1.6: Syntax und Werkzeuge

a) Zuerst meldet Python die fehlende Einrückung – noch vor dem Start, deshalb erscheint der `NameError` nicht:

```
  File "s1_6.py", line 4
    print("eingerückt?")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 3
```

Ist Zeile 4 eingerückt, läuft das Programm bis Zeile 2 und meldet dann (Groß-/Kleinschreibung zählt):

```
Traceback (most recent call last):
  File "s1_6b.py", line 2, in <module>
    Print("Hallo DHBW")
    ^^^^^
NameError: name 'Print' is not defined. Did you mean: 'print'?
```

b) `pwd` zeigt den aktuellen Ordner, `cd ..` wechselt eine Ebene nach oben, `uv run hallo.py` führt das Programm in der Projektumgebung aus, `ruff check` findet typische Fehler und Stilprobleme, `ty check` prüft die Datentypen, ohne das Programm auszuführen.

Aufgabenpool VL2 – Lösungen

Lösung 2.1: Datentypen

Ausgabe (geprüft mit Python 3.14):

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
(3, 5) <class 'tuple'>
```

Achtung: `3,5` ist ein Tupel aus zwei Zahlen, keine Kommazahl – Dezimaltrenner ist der Punkt.

Lösung 2.2: Namen und Konventionen

```
kunden_anzahl   gueltig=True
2tes_konto      gueltig=False
MWST_SATZ       gueltig=True
kundenAnzahl    gueltig=True
netto-preis     gueltig=False
class           gueltig=False
groesse         gueltig=True
Kunde           gueltig=True
```

Name	Bewertung
`kunden_anzahl`	gültig, snake_case: Variable/Funktion
`2tes_konto`	ungültig: beginnt mit Ziffer
`MWST_SATZ`	gültig, UPPER_SNAKE_CASE: Konstante (mit `Final`)
`kundenAnzahl`	gültig, aber camelCase (Java-Stil) – ruff meldet (N)
`netto-preis`	ungültig: `-` ist Minus (kebab-case)
`class`	ungültig: Schlüsselwort
`groesse`	gültig, snake_case, ohne Umlaut
`Kunde`	gültig, PascalCase: Klasse

Lösung 2.3: Zuweisungen nachvollziehen

Ausgabe (geprüft mit Python 3.14):

```
9
3 5
7
```

x: 5 → 15 → 14 → 3 → 9. a, b: (1, 2) → (2, 3) → (3, 5). `++y` ist zweimal das Vorzeichen +, ändert nichts (ruff B002); `y++` wäre ein `SyntaxError`.

Lösung 2.4: Euro in Cent

Ausgabe (geprüft mit Python 3.14):

```
999
1998
1998.9999999999998
1999
1999
```

`19.99 * 100` ist als `float` 1998.9999…, `int()` schneidet ab → ein Cent fehlt. Abhilfen: `round()` oder `Decimal("19.99")` (als Text übergeben).

Lösung 2.5: f-Strings und Umwandlungen

Ausgabe (geprüft mit Python 3.14):

```
7.50 €
Ada ist 21 Jahre alt
Brutto: 59.38
AdaAda|3|Aa
-3 5.0 421
```

`int(-3.7)` schneidet ab (Richtung 0), `str(42) + "1"` verkettet Texte.

Lösung 2.6: Fehlerarten erkennen

a)

```
  File "p2_fehler4.py", line 2
    print(f"Die Zahl ist {zahl}"
          ^
SyntaxError: '(' was never closed
```

b)

```
Traceback (most recent call last):
  File "p2_fehler1.py", line 2, in <module>
    print("Alter: " + alter)
          ~~~~~~~~~~^~~~~~~
TypeError: can only concatenate str (not "int") to str
```

c)

```
Traceback (most recent call last):
  File "p2_fehler2.py", line 3, in <module>
    print(a / b)
          ~~^~~
ZeroDivisionError: division by zero
```

d)

```
Traceback (most recent call last):
  File "p2_fehler6.py", line 2, in <module>
    print(anzahl)
          ^^^^^^
NameError: name 'anzahl' is not defined
```

e)

```
Traceback (most recent call last):
  File "p2_fehler5.py", line 1, in <module>
    print(int("12.5"))
          ~~~^^^^^^^^
ValueError: invalid literal for int() with base 10: '12.5'
```

f)

```
  File "p2_fehler3.py", line 2
    x++
       ^
SyntaxError: invalid syntax
```

a) und f) Syntaxfehler (vor dem Start), b) Typfehler (`ty` vorher, sonst `TypeError` beim Ausführen), c) Laufzeitfehler, d) `NameError`: eine bloße Typangabe legt keine Variable an, e) Laufzeitfehler `ValueError` (ungültiger Text für `int`). Traceback von unten nach oben lesen.

Lösung 2.7: Mini-Rechner und Jahreszahl ohne `int()`

```
# Teil a: Mini-Rechner
a = "12"
b = "3"
ai = int(a)
bi = int(b)
print(f"a={ai}, b={bi} | sum={ai + bi}, diff={ai - bi}, prod={ai * bi}, quot={ai / bi}")
print(a + b, a * 3)

# Teil b: Jahreszahl ohne int()
jahr = "2026"
null = ord("0")
zahl = (
    (ord(jahr[0]) - null) * 1000
    + (ord(jahr[1]) - null) * 100
    + (ord(jahr[2]) - null) * 10
    + (ord(jahr[3]) - null)
)
print(zahl, zahl + 1)
```

Ausgabe (geprüft mit Python 3.14):

```
a=12, b=3 | sum=15, diff=9, prod=36, quot=4.0
123 121212
2026 2027
```

Aufgabenpool VL3 – Lösungen

Lösung 3.1: Noten mit `if/elif`

Ausgabe (geprüft mit Python 3.14):

```
ausreichend
```

`elif` prüft nacheinander und stoppt beim ersten Treffer: 95 ≥ 50 trifft sofort (Logikfehler ohne Fehlermeldung). Größte Grenze zuerst:

```
def note(punkte: int) -> str:
    if punkte >= 90:
        return "sehr gut"
    elif punkte >= 75:
        return "gut"
    elif punkte >= 60:
        return "befriedigend"
    elif punkte >= 50:
        return "ausreichend"
    else:
        return "nicht bestanden"


for p in [95, 90, 89, 75, 74, 60, 59, 50, 49]:
    print(p, note(p))
```

Ausgabe (geprüft mit Python 3.14):

```
95 sehr gut
90 sehr gut
89 gut
75 gut
74 befriedigend
60 befriedigend
59 ausreichend
50 ausreichend
49 nicht bestanden
```

Testwerte: jeweils auf und knapp unter jeder Grenze.

Lösung 3.2: Kurzschlussauswertung

Ausgabe (geprüft mit Python 3.14):

```
x != 0 and 10 / x > 1 -> False
10 / x > 1 and x != 0 -> ZeroDivisionError: division by zero
x == 0 or 10 / x > 1 -> True
x > 0 or 10 / x > 1 -> ZeroDivisionError: division by zero
```

Python wertet von links nach rechts aus und hört auf, sobald das Ergebnis feststeht. Bei a) ist `and` nach `False` entschieden, bei c) `or` nach `True` – Absicherung gehört nach vorn.

Lösung 3.3: Wahrheitswerte und bedingter Ausdruck

Ausgabe (geprüft mit Python 3.14):

```
nein ja nein ja nein ja ja nein
True
False
True
```

Falsy: `[]`, `0`, `None`, `""`. Wahr: `"0"`, `" "`, `[0]`, `-1`. Rangfolge: `not` vor `and` vor `or`; `a != b` ist XOR.

Lösung 3.4: Schleifen: `range`, `break`, `continue`

Ausgabe (geprüft mit Python 3.14):

```
5 3 1
1 5 9 13 17
leer?
1 2 4 5 7 8
5 16 8 4 2 1
10 20 21
```

`range(10, 3)` ist leer (Startwert schon ≥ stop, kein Fehler). Bei der `while`-Schleife ist die Zahl der Durchläufe vorher unbekannt.

Lösung 3.5: Parkgebühren mit `match`

Ausgabe (geprüft mit Python 3.14):

```
Transporter, 6 h: 18.30 Euro
PKW, 12 h: 23.10 Euro
PKW, 9 h: 19.50 Euro
Motorrad, 60 h: 50.00 Euro
Unbekannter Fahrzeugtyp: LKW
```

Transporter: 6 · 3,50 = 21,00, −20 % = 16,80, + 1,50 = 18,30. PKW 12 h: 24 · 0,9 = 21,60 + 1,50. Motorrad: 61,50 → Deckel 50. Reihenfolge: Rabatt → Grundgebühr → Deckel. Kein `break` nötig, kein Durchfallen; `case _` fängt alles andere.

Lösung 3.6: Schaltjahr

```
def ist_schaltjahr(jahr: int) -> bool:
    return jahr % 4 == 0 and (jahr % 100 != 0 or jahr % 400 == 0)


for jahr in [1900, 2000, 2024, 2026]:
    print(jahr, ist_schaltjahr(jahr))
```

Ausgabe (geprüft mit Python 3.14):

```
1900 False
2000 True
2024 True
2026 False
```

Lösung 3.7: Temperaturliste

```
wochentage = ["Mo", "Di", "Mi", "Do", "Fr"]
temperaturen = [-3.5, -1.0, -4.5, -2.0, -0.5]

summe = 0.0
maximum = temperaturen[0]  # nicht 0, sonst falsch bei lauter negativen Werten
for tag, temp in zip(wochentage, temperaturen, strict=True):
    print(f"{tag}: {temp} Grad")
    summe += temp
    if temp > maximum:
        maximum = temp
print(f"Durchschnitt: {summe / len(temperaturen):.2f} Grad, Maximum: {maximum} Grad")

# Kurzform mit eingebauten Funktionen
print(f"Durchschnitt: {sum(temperaturen) / len(temperaturen):.2f} Grad, Maximum:
{max(temperaturen)} Grad")
```

Ausgabe (geprüft mit Python 3.14):

```
Mo: -3.5 Grad
Di: -1.0 Grad
Mi: -4.5 Grad
Do: -2.0 Grad
Fr: -0.5 Grad
Durchschnitt: -2.30 Grad, Maximum: -0.5 Grad
Durchschnitt: -2.30 Grad, Maximum: -0.5 Grad
```

Mit Startwert 0 käme bei lauter negativen Werten fälschlich 0 heraus.

Lösung 3.8: Listenfalle und Sieb des Eratosthenes

a)

Ausgabe (geprüft mit Python 3.14):

```
[[1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0]]
[1.0, 3.7] 1.7 [1.0, 1.7, 2.3, 3.7] [2.3, 1.0, 3.7, 1.7]
```

`[[0] * 3] * 2` kopiert nur den Verweis: beide Zeilen sind **dieselbe** Liste. Richtig: Zeilen in einer Schleife einzeln anlegen. `sorted` liefert eine sortierte Kopie.

b)

```
from typing import Final

N: Final = 30

ist_prim = [True] * (N + 1)
ist_prim[0] = False
ist_prim[1] = False
p = 2
while p * p <= N:
    if ist_prim[p]:
        for vielfaches in range(p * p, N + 1, p):
            ist_prim[vielfaches] = False
    p += 1

for zahl, prim in enumerate(ist_prim):
    if prim:
        print(zahl, end=" ")
print()
```

Ausgabe (geprüft mit Python 3.14):

```
2 3 5 7 11 13 17 19 23 29
```

Abbruch ab `p * p > N`; Streichen ab `p * p` (kleinere Vielfache sind schon gestrichen).

Aufgabenpool VL4 – Lösungen

Lösung 4.1: Standardwerte und Schlüsselwortargumente

a)

Ausgabe (geprüft mit Python 3.14):

```
119.0
107.0
107.0
23.79
Traceback (most recent call last):
  File "s4_1.py", line 9, in <module>
    print(brutto())
          ~~~~~~^^
TypeError: brutto() missing 1 required positional argument: 'netto'
```

`brutto(100)` nutzt den Standardwert 0.19; Position und Schlüsselwortargumente liefern beide 107.0 (bei Schlüsselwörtern ist die Reihenfolge egal); 19.99 · 1.19 wird auf 23.79 gerundet. Fehlt ein Pflichtparameter, läuft die Funktion gar nicht erst los.

b)

```
  File "s4_1b.py", line 1
    def f(a: int = 1, b: int) -> int:
                      ^^^^^^
SyntaxError: parameter without a default follows parameter with a default
```

Parameter mit Standardwert müssen hinter denen ohne stehen.

Lösung 4.2: Gültigkeitsbereiche

Links:

```
in f: 3
Traceback (most recent call last):
  File "s4_2a.py", line 7, in <module>
    print(y)
          ^
NameError: name 'y' is not defined
```

Rechts:

```
In der Funktion: 6
Nach dem Aufruf: 5
```

Unten:

```
119.0
Traceback (most recent call last):
  File "s4_2b.py", line 14, in <module>
    erhoehe()
    ~~~~~~~^^
  File "s4_2b.py", line 10, in erhoehe
    zaehler += 1
    ^^^^^^^
UnboundLocalError: cannot access local variable 'zaehler' where it is not associated with a
value
```

Lokale Variablen existieren nur während des Aufrufs. Globale Werte lesen geht (LEGB), eine Zuweisung (`+=`) macht den Namen lokal → `UnboundLocalError`. `wert = wert + 1` bindet nur das

lokale Namensschild neu.

Lösung 4.3: `break` oder `return`?

Ausgabe (geprüft mit Python 3.14):

```
1 2 3 Schleife beendet.
1 2 3 | fertig
```

`break` verlässt nur die Schleife, der Rest der Funktion läuft. `return` verlässt die ganze Funktion – „Schleife beendet.“ erscheint nicht.

Lösung 4.4: Veränderbarer Standardwert

Ausgabe (geprüft mit Python 3.14):

```
[1, 2] [1, 2] [0, 3]
True
```

Die Standardliste wird einmal beim Definieren erzeugt und bei jedem Aufruf ohne zweites Argument wiederverwendet (ruff B006). Richtig:

```
def anhaengen(wert: int, liste: list[int] | None = None) -> list[int]:
    if liste is None:
        liste = []
    liste.append(wert)
    return liste


a = anhaengen(1)
b = anhaengen(2)
c = anhaengen(3, [0])
print(a, b, c)
print(a is b)
```

Ausgabe (geprüft mit Python 3.14):

```
[1] [2] [0, 3]
False
```

Lösung 4.5: Tupel als Rückgabe und `*args`

a)

Ausgabe (geprüft mit Python 3.14):

```
2 9
(5, 5)
10 0
```

„Mehrere Rückgabewerte“ sind ein Tupel; `*zahlen` sammelt alle Argumente in einem Tupel (ohne Argumente: `()`).

b)

```
def durchschnitt(*werte: float) -> float:
    """Berechnet den Durchschnitt beliebig vieler Werte.

    Args:
        werte: Beliebig viele Zahlen.

    Returns:
        Den Durchschnitt, 0.0 wenn keine Werte übergeben wurden.
    """
    if len(werte) == 0:
        return 0.0
    gesamt = 0.0
    for wert in werte:
        gesamt += wert
    return gesamt / len(werte)


print(durchschnitt(2, 4, 9), durchschnitt(), durchschnitt(1.5))
```

Ausgabe (geprüft mit Python 3.14):

```
5.0 0.0 1.5
```

Lösung 4.6: Rekursion nachvollziehen

Ausgabe (geprüft mit Python 3.14):

```
10
1 2 3 4
```

Abstieg: `summe_bis(4)` wartet auf `summe_bis(3)` … bis `summe_bis(0) = 0`. Aufstieg: 1 + 0 = 1, 2 + 1 = 3, 3 + 3 = 6, 4 + 6 = 10. Bei `raetsel` steht `print` **nach** dem rekursiven Aufruf: Ausgegeben wird erst im Aufstieg, also beginnend bei 1.

Lösung 4.7: Rekursive Funktionen schreiben

```
def zaehle_zeichen(text: str, zeichen: str) -> int:
    """Zählt rekursiv, wie oft zeichen in text vorkommt."""
    if text == "":
        return 0
    treffer = 1 if text[0] == zeichen else 0
    return treffer + zaehle_zeichen(text[1:], zeichen)


def potenz(basis: float, exponent: int) -> float:
    """Berechnet basis hoch exponent rekursiv, auch für negative Exponenten."""
    if exponent == 0:
        return 1.0
    if exponent < 0:
        return 1 / potenz(basis, -exponent)
    return basis * potenz(basis, exponent - 1)


def maximum(zahlen: list[int], index: int = 0) -> int:
    """Liefert rekursiv das größte Element einer nicht leeren Liste."""
    if index == len(zahlen) - 1:
        return zahlen[index]
    rest = maximum(zahlen, index + 1)
    return zahlen[index] if zahlen[index] > rest else rest


def main() -> None:
    print(zaehle_zeichen("banana", "a"), zaehle_zeichen("", "a"))
    print(potenz(2, 4), potenz(2, -2), potenz(5, 0))
    print(maximum([4, 9, 2, 7, 5]), maximum([3]), maximum([-5, -2, -9]))


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
3 0
16.0 0.25 1.0
9 3 -2
```

Achtung: `maximum([])` erreicht die Abbruchbedingung nie (`index == -1`) – leere Listen sind ausgeschlossen. Tests mit `float`: `potenz(0.1, 3)` ergibt 0.0010000000000000002, nicht mit `== 0.001` prüfen.

Lösung 4.8: Applikativ programmieren und testen

```
def h(p: bool, q: bool, x: int, y: int) -> int:
    """h(p, q, x, y) = if p und q then 4x + 2y else 5x - 3y."""
    return 4 * x + 2 * y if p and q else 5 * x - 3 * y


def fibonacci(n: int) -> int:
    """Liefert die n-te Fibonacci-Zahl rekursiv (fibonacci(0) = 0)."""
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)


if __name__ == "__main__":
    print(h(True, False, 1, 3), h(True, True, 2, 8), fibonacci(10), fibonacci(20))
```

Ausgabe (geprüft mit Python 3.14):

```
-4 24 55 6765
```

```
from applikativ import fibonacci, h


def test_h_else_zweig() -> None:
    assert h(True, False, 1, 3) == -4


def test_h_then_zweig() -> None:
    assert h(True, True, 2, 8) == 24


def test_h_beide_falsch() -> None:
    assert h(False, False, 2, 1) == 7


def test_fibonacci_anfang() -> None:
    assert fibonacci(0) == 0
    assert fibonacci(1) == 1


def test_fibonacci_zehn() -> None:
    assert fibonacci(10) == 55
```

Geprüft ohne pytest: `test_applikativ: 5 Tests bestanden`

d) Dieselben Werte werden immer wieder neu berechnet; die Zahl der Aufrufe wächst exponentiell (für n = 10 schon 177 Aufrufe, selbst nachgezählt; allgemein 2·fib(n+1) − 1). Abhilfe laut VL04: `@functools.cache` oder iterativ.

Aufgabenpool VL5 – Lösungen

Lösung 5.1: Grundbegriffe und UML

a) Die **Klasse** ist der Bauplan (`Mitarbeiter`), ein **Objekt** ein danach gebautes Exemplar mit eigenen Werten (Lena, 4200 €). **Attribute** beschreiben den Zustand (`name`, `monatsgehalt`), **Methoden** das Verhalten (`gehalt_erhoehen()`, `abteilung_wechseln()`). UML: Kasten mit Name oben, Attributen in der Mitte, Methoden unten.

b)

```
Traceback (most recent call last):
  File "s5_1_ohne_self.py", line 6, in <module>
    kaffee = Produkt("Kaffee", 12.99)
TypeError: Produkt.__init__() takes 2 positional arguments but 3 were given
```

Python übergibt das neue Objekt automatisch als erstes Argument. Ohne `self` sind es drei Argumente für zwei Parameter. (Außerdem wäre `self` im Rumpf unbekannt.)

Lösung 5.2: Methoden und `__str__`/`__repr__`

Ausgabe (geprüft mit Python 3.14):

```
Abgelehnt: Der Kontostand reicht nicht.
False
True
Konto von Anna: 120 Euro
[Konto('Anna', 120), Konto('Ben', 25)]
```

100 + 50 = 150, Abheben von 500 abgelehnt, 30 abgehoben → 120. `Konto.einzahlen(ben, 25)` ist dasselbe wie `ben.einzahlen(25)`. `print` nutzt `__str__`, eine Liste zeigt ihre Elemente mit `__repr__`.

Lösung 5.3: Namen, Identität und Gleichheit

Ausgabe (geprüft mit Python 3.14):

```
150
True False
150 0
True False
Kunde(name='Anna Berger', ort='Heilbronn')
```

`konto2 = konto1` erzeugt kein neues Konto, sondern einen zweiten Namen. `==` vergleicht bei normalen Klassen die Identität, bei `@dataclass` die Werte. Die Neuzuweisung `konto2 = Konto(...)` ändert `konto1` nicht.

Lösung 5.4: Klassenattribute und Tippfehler

Ausgabe (geprüft mit Python 3.14):

```
K001 K002
2 2 3
12 12 3
0
```

`ben.anzahl_konten += 1` liest das Klassenattribut und legt ein neues **Instanzattribut** nur für `ben` an; die Klasse bleibt bei 2. Klassenattribute immer über den Klassennamen ändern. Der

Tippfehler `kontostnad` legt zur Laufzeit stillschweigend ein neues Attribut an; `ty` meldet `error[unresolved-attribute]`.

Lösung 5.5: Klasse `Produkt` schreiben

```
from decimal import Decimal


class Produkt:
    def __init__(self, name: str, preis: Decimal, bestand: int) -> None:
        self.name = name
        self.preis = preis
        self.bestand = bestand

    def lagerwert(self) -> Decimal:
        return self.preis * self.bestand

    def einlagern(self, menge: int) -> None:
        if menge <= 0:
            print(f"Ungültige Menge: {menge}")
            return
        self.bestand += menge

    def verkaufen(self, menge: int) -> bool:
        if menge > self.bestand:
            print(f"Nicht genug {self.name} auf Lager (Bestand {self.bestand}).")
            return False
        self.bestand -= menge
        return True

    def __str__(self) -> str:
        return f"{self.name}: {self.preis} Euro, Bestand {self.bestand}"

    def __repr__(self) -> str:
        return f"Produkt({self.name!r}, Decimal('{self.preis}'), {self.bestand})"


def main() -> None:
    kaffee = Produkt("Kaffee 1 kg", Decimal("12.99"), 40)
    tee = Produkt("Grüner Tee", Decimal("3.49"), 120)
    print(kaffee.verkaufen(5))
    print(kaffee.verkaufen(100))
    kaffee.einlagern(10)
    kaffee.einlagern(-3)
    print(kaffee)
    print(kaffee.lagerwert())
    print([kaffee, tee])


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
True
Nicht genug Kaffee 1 kg auf Lager (Bestand 35).
False
Ungültige Menge: -3
Kaffee 1 kg: 12.99 Euro, Bestand 45
584.55
[Produkt('Kaffee 1 kg', Decimal('12.99'), 45), Produkt('Grüner Tee', Decimal('3.49'), 120)]
```

Lösung 5.6: Kapselung mit Properties

```
from decimal import Decimal
from typing import Final

MWST_SATZ: Final = Decimal("0.19")


class Produkt:
    def __init__(self, name: str, preis: Decimal, bestand: int) -> None:
        self.name = name
        self._preis = Decimal(0)
        self.preis = preis  # nutzt den Setter
        self._bestand = bestand

    @property
    def bestand(self) -> int:
        return self._bestand

    @property
    def preis(self) -> Decimal:
        return self._preis

    @preis.setter
    def preis(self, neuer_preis: Decimal) -> None:
        if neuer_preis < 0:
            print(f"Ungültiger Preis: {neuer_preis}")
            return
        self._preis = neuer_preis

    @property
    def bruttopreis(self) -> Decimal:
        return (self._preis * (1 + MWST_SATZ)).quantize(Decimal("0.01"))

    def verkaufen(self, menge: int) -> bool:
        if menge > self._bestand:
            return False
        self._bestand -= menge
        return True


def main() -> None:
    kaffee = Produkt("Kaffee 1 kg", Decimal("12.99"), 40)
    print(kaffee.bruttopreis)
    kaffee.preis = Decimal("-5")
    print(kaffee.preis)
    kaffee.verkaufen(5)
    print(kaffee.bestand)
    kaffee.bestand = 100


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
15.46
Ungültiger Preis: -5
12.99
35
Traceback (most recent call last):
  File "s5_6.py", line 51, in <module>
    main()
    ~~~~^^
  File "s5_6.py", line 47, in main
    kaffee.bestand = 100
    ^^^^^^^^^^^^^^
AttributeError: property 'bestand' of 'Produkt' object has no setter
```

`ty check` meldet die letzte Zeile schon vorher als Zuweisung an eine read-only property.

Lösung 5.7: Warenkorb: Hat-ein-Beziehung und `@dataclass`

```
from dataclasses import dataclass
from decimal import Decimal


class Produkt:
    def __init__(self, name: str, preis: Decimal, bestand: int) -> None:
        self.name = name
        self._preis = preis
        self._bestand = bestand

    @property
    def preis(self) -> Decimal:
        return self._preis

    @property
    def bestand(self) -> int:
        return self._bestand


@dataclass
class Kunde:
    name: str
    email: str


@dataclass
class Position:
    produkt: Produkt
    menge: int

    def summe(self) -> Decimal:
        return self.produkt.preis * self.menge


class Warenkorb:
    def __init__(self, kunde: Kunde) -> None:
        self.kunde = kunde
        self.positionen: list[Position] = []

    def hinzufuegen(self, produkt: Produkt, menge: int) -> None:
        if menge > produkt.bestand:
            print(f"Nur noch {produkt.bestand} x {produkt.name} auf Lager.")
            return
        self.positionen.append(Position(produkt, menge))

    def gesamtsumme(self) -> Decimal:
        summe = Decimal(0)
        for position in self.positionen:
            summe += position.summe()
        return summe

    def ausgeben(self) -> None:
        print(f"Warenkorb von {self.kunde.name}")
        for p in self.positionen:
            print(f"{p.menge} x {p.produkt.name}: {p.summe()} Euro")
        print(f"Summe: {self.gesamtsumme()} Euro")


def main() -> None:
    anna = Kunde("Anna Berger", "anna.berger@example.com")
    kaffee = Produkt("Kaffee 1 kg", Decimal("12.99"), 40)
```

```
    tee = Produkt("Grüner Tee", Decimal("3.49"), 120)
    korb = Warenkorb(anna)
    korb.hinzufuegen(kaffee, 2)
    korb.hinzufuegen(tee, 3)
    korb.hinzufuegen(kaffee, 50)
    korb.ausgeben()


if __name__ == "__main__":
    main()
```

Ausgabe (geprüft mit Python 3.14):

```
Nur noch 40 x Kaffee 1 kg auf Lager.
Warenkorb von Anna Berger
2 x Kaffee 1 kg: 25.98 Euro
3 x Grüner Tee: 10.47 Euro
Summe: 36.45 Euro
```

`Kunde` und `Position` bündeln vor allem Daten → `@dataclass`. Beim `Warenkorb` stehen Verhalten und Regeln im Vordergrund (Bestandsprüfung) → normale Klasse.